

DBMS Joins & Transactions Lab Report

Complete Screenshot Record & Query Execution Log (13 Screenshots)

1. Step 1: Database Creation & Initial Table Definitions

Initializes the lab database environment and creates the core entity tables along with primary keys and foreign key relationships.

```
mysql> DROP DATABASE IF EXISTS DBMS_Joins_Transactions;
Query OK, 0 rows affected, 1 warning (0.02 sec)

mysql>
mysql> CREATE DATABASE DBMS_Joins_Transactions;
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> USE DBMS_Joins_Transactions;
Database changed
mysql> CREATE TABLE roles (
  ->   role_id INT PRIMARY KEY,
  ->   role_name VARCHAR(50)
  -> );
Query OK, 0 rows affected (0.06 sec)

mysql>
mysql> CREATE TABLE users (
  ->   user_id INT PRIMARY KEY,
  ->   name VARCHAR(50),
  ->   email VARCHAR(50),
  ->   role_id INT
  -> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> CREATE TABLE departments (
  ->   dept_id INT PRIMARY KEY,
  ->   dept_name VARCHAR(50)
  -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> CREATE TABLE employees (
  ->   emp_id INT PRIMARY KEY,
  ->   emp_name VARCHAR(50),
  ->   dept_id INT,
  ->   manager_id INT,
  ->   salary DECIMAL(10,2)
  -> );
```

Screenshot 1: Database creation (`DBMS\_Joins\_Transactions`) and schema definition for `roles`, `users`, `departments`, and `employees` tables.

2. Step 2: Table Schema Definition (Customers, Orders, Accounts)

Completes table structures for transactional entities and populates reference tables (`roles` and `users`).

```
Query OK, 0 rows affected (0.07 sec)

mysql>
mysql> CREATE TABLE customers (
->   customer_id INT PRIMARY KEY,
->   customer_name VARCHAR(50),
->   city VARCHAR(50)
-> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> CREATE TABLE orders (
->   order_id INT PRIMARY KEY,
->   customer_id INT,
->   order_date DATE,
->   total_amount DECIMAL(10,2)
-> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> CREATE TABLE accounts (
->   account_no INT PRIMARY KEY,
->   holder_name VARCHAR(50),
->   balance DECIMAL(10,2)
-> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO roles VALUES
-> (1, 'Admin'),
-> (2, 'Manager'),
-> (3, 'Employee'),
-> (4, 'Guest');
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> INSERT INTO users VALUES
-> (101, 'Rahul', 'rahul@gmail.com', 1),
-> (102, 'Arjun', 'arjun@gmail.com', 2),
-> (103, 'Priya', 'priya@gmail.com', 3),
-> (104, 'Sneha', 'sneha@gmail.com', NULL);
Query OK, 4 rows affected (0.01 sec)
```

Screenshot 2: Schema definition for `customers`, `orders`, and `accounts` tables, followed by initial data insertions into `roles` and `users`.

3. Step 3: Populating Core Lab Data (Departments, Employees, Customers, Orders)

Inserts test datasets across multiple related tables to facilitate join queries and aggregate calculations.

```
mysql> INSERT INTO departments VALUES
-> (1, 'CSE'),
-> (2, 'ECE'),
-> (3, 'EEE'),
-> (4, 'MECH');
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> INSERT INTO employees VALUES
-> (1, 'Rahul', 1, NULL, 80000),
-> (2, 'Arjun', 1, 1, 65000),
-> (3, 'Priya', 1, 1, 55000),
-> (4, 'Sneha', 2, 1, 60000),
-> (5, 'Kiran', 2, 1, 70000),
-> (6, 'Anil', 3, 2, 45000),
-> (7, 'Vijay', 1, 1, 90000),
-> (8, 'Meena', 3, 2, 50000);
Query OK, 8 rows affected (0.01 sec)
Records: 8 Duplicates: 0 Warnings: 0

mysql> INSERT INTO customers VALUES
-> (1, 'Customer1', 'Hyderabad'),
-> (2, 'Customer2', 'Mumbai'),
-> (3, 'Customer3', 'Delhi'),
-> (4, 'Customer4', 'Chennai');
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> INSERT INTO orders VALUES
-> (101, 1, '2026-01-10', 50000),
-> (102, 2, '2026-01-15', 30000),
-> (103, 1, '2026-02-05', 20000),
-> (104, 3, '2026-02-20', 45000),
-> (105, 2, '2026-03-10', 25000),
-> (106, 1, '2026-03-15', 15000);
Query OK, 6 rows affected (0.01 sec)
Records: 6 Duplicates: 0 Warnings: 0
```

Screenshot 3: Data insertion queries into `departments`, `employees`, `customers`, and `orders` tables.

4. Step 4: Populating Accounts & Database Table Verification

Verifies that all 7 required tables are properly registered in MySQL and checks initial records for user roles.

```
mysql> INSERT INTO accounts VALUES
  -> (101, 'Rahul', 20000),
  -> (102, 'Arjun', 30000),
  -> (103, 'Priya', 15000);
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SHOW TABLES;
+-----+
| Tables_in_dbms_joins_transactions |
+-----+
| accounts                          |
| customers                         |
| departments                       |
| employees                        |
| orders                           |
| roles                             |
| users                             |
+-----+
7 rows in set (0.01 sec)

mysql> SELECT * FROM roles;
+-----+-----+
| role_id | role_name |
+-----+-----+
| 1       | Admin    |
| 2       | Manager  |
| 3       | Employee |
| 4       | Guest    |
+-----+-----+
4 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM users;
+-----+-----+-----+-----+
| user_id | name  | email                | role_id |
+-----+-----+-----+-----+
| 101     | Rahul | rahul@gmail.com      | 1       |
| 102     | Arjun | arjun@gmail.com      | 2       |
| 103     | Priya | priya@gmail.com      | 3       |
+-----+-----+-----+-----+
```

Screenshot 4: Insertion into `accounts`, `SHOW TABLES` listing, and verifying `roles` and `users` contents.

5. Step 5: Verifying Departments, Employees, and Customers Tables

Ensures all employee records (8 rows) and department associations are stored correctly prior to performing join operations.

```
mysql>
mysql> SELECT * FROM departments;
+-----+-----+
| dept_id | dept_name |
+-----+-----+
| 1 | CSE |
| 2 | ECE |
| 3 | EEE |
| 4 | MECH |
+-----+-----+
4 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM employees;
+-----+-----+-----+-----+-----+
| emp_id | emp_name | dept_id | manager_id | salary |
+-----+-----+-----+-----+-----+
| 1 | Rahul | 1 | NULL | 80000.00 |
| 2 | Arjun | 1 | 1 | 65000.00 |
| 3 | Priya | 1 | 1 | 55000.00 |
| 4 | Sneha | 2 | 1 | 60000.00 |
| 5 | Kiran | 2 | 1 | 70000.00 |
| 6 | Anil | 3 | 2 | 45000.00 |
| 7 | Vijay | 1 | 1 | 90000.00 |
| 8 | Meena | 3 | 2 | 50000.00 |
+-----+-----+-----+-----+-----+
8 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM customers;
+-----+-----+-----+
| customer_id | customer_name | city |
+-----+-----+-----+
| 1 | Customer1 | Hyderabad |
| 2 | Customer2 | Mumbai |
| 3 | Customer3 | Delhi |
| 4 | Customer4 | Chennai |
+-----+-----+-----+
4 rows in set (0.00 sec)
```

Screenshot 5: Full select queries verifying data in `departments`, `employees`, and `customers` tables.

6. Step 6: Verifying Orders, Accounts & Executing Query Q1 (INNER JOIN)

Executes Query Q1: Retrieves employee names paired with their respective department names using `INNER JOIN`.

```
mysql>
mysql> SELECT * FROM orders;
+-----+-----+-----+-----+
| order_id | customer_id | order_date | total_amount |
+-----+-----+-----+-----+
| 101 | 1 | 2026-01-10 | 50000.00 |
| 102 | 2 | 2026-01-15 | 30000.00 |
| 103 | 1 | 2026-02-05 | 20000.00 |
| 104 | 3 | 2026-02-20 | 45000.00 |
| 105 | 2 | 2026-03-10 | 25000.00 |
| 106 | 1 | 2026-03-15 | 15000.00 |
+-----+-----+-----+-----+
6 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM accounts;
+-----+-----+-----+
| account_no | holder_name | balance |
+-----+-----+-----+
| 101 | Rahul | 20000.00 |
| 102 | Arjun | 30000.00 |
| 103 | Priya | 15000.00 |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> -- Q1
mysql> SELECT emp_name, dept_name
-> FROM employees e
-> INNER JOIN departments d
-> ON e.dept_id = d.dept_id;
+-----+-----+
| emp_name | dept_name |
+-----+-----+
| Rahul | CSE |
| Arjun | CSE |
| Priya | CSE |
| Sneha | ECE |
| Kiran | ECE |
| Anil | EEE |
| Vijay | CSE |
+-----+-----+
```

Screenshot 6: Verifying `orders` and `accounts`, followed by Query Q1 (INNER JOIN between Employees and Departments).

7. Step 7: Queries Q2 (LEFT JOIN), Q3 (RIGHT JOIN), and Q4 (FULL OUTER JOIN Start)

Demonstrates `LEFT JOIN` (including unmatched user records with NULL roles) and `RIGHT JOIN` (including unmatched roles).

```

mysql>
mysql> -- Q2
mysql> SELECT name, email, role_name
  -> FROM users u
  -> LEFT JOIN roles r
  -> ON u.role_id = r.role_id;
+-----+-----+-----+
| name | email | role_name |
+-----+-----+-----+
| Rahul | rahul@gmail.com | Admin |
| Arjun | arjun@gmail.com | Manager |
| Priya | priya@gmail.com | Employee |
| Sneha | sneha@gmail.com | NULL |
+-----+-----+-----+
4 rows in set (0.00 sec)

mysql>
mysql>
mysql> -- Q3
mysql> SELECT name, role_name
  -> FROM users u
  -> RIGHT JOIN roles r
  -> ON u.role_id = r.role_id;
+-----+-----+
| name | role_name |
+-----+-----+
| Rahul | Admin |
| Arjun | Manager |
| Priya | Employee |
| NULL | Guest |
+-----+-----+
4 rows in set (0.00 sec)

mysql>
mysql>
mysql> -- Q4
mysql> SELECT name, email, role_name
  -> FROM users u
  -> LEFT JOIN roles r
  -> ON u.role_id = r.role_id
  -> UNION

```

Screenshot 7: Queries Q2, Q3, and beginning of Q4 testing LEFT, RIGHT, and emulated FULL OUTER JOINS.

8. Step 8: Completing Q4 (UNION) & Query Q5 (SELF JOIN)

Q4 combines LEFT and RIGHT joins using `UNION`. Q5 performs a `SELF JOIN` mapping each employee to their respective manager.

```

-> UNION
-> SELECT name, email, role_name
-> FROM users u
-> RIGHT JOIN roles r
-> ON u.role_id = r.role_id;

```

name	email	role_name
Rahul	rahul@gmail.com	Admin
Arjun	arjun@gmail.com	Manager
Priya	priya@gmail.com	Employee
Sneha	sneha@gmail.com	NULL
NULL	NULL	Guest

5 rows in set (0.01 sec)

```

mysql>
mysql>
mysql> -- Q5
mysql> SELECT e.emp_name AS Employee,
->         m.emp_name AS Manager
-> FROM employees e
-> LEFT JOIN employees m
-> ON e.manager_id = m.emp_id;

```

Employee	Manager
Rahul	NULL
Arjun	Rahul
Priya	Rahul
Sneha	Rahul
Kiran	Rahul
Anil	Arjun
Vijay	Rahul
Meena	Arjun

8 rows in set (0.00 sec)

Screenshot 8: Results of Q4 (FULL OUTER JOIN via UNION) and Query Q5 (SELF JOIN on Employees for Manager hierarchy).

9. Step 9: Queries Q6 (Customer Orders JOIN) & Q7 (Department Salary Aggregation)

Q6 lists customer purchases with order details. Q7 calculates `COUNT(\*)`, `AVG(salary)`, and `MAX(salary)` grouped by department.

```

mysql>
mysql> -- Q6
mysql> SELECT o.order_id,
->         customer_name,
->         city,
->         total_amount
-> FROM customers c
-> JOIN orders o
-> ON c.customer_id = o.customer_id;
+-----+-----+-----+-----+
| order_id | customer_name | city      | total_amount |
+-----+-----+-----+-----+
| 101      | Customer1     | Hyderabad | 50000.00     |
| 102      | Customer2     | Mumbai    | 30000.00     |
| 103      | Customer1     | Hyderabad | 20000.00     |
| 104      | Customer3     | Delhi     | 45000.00     |
| 105      | Customer2     | Mumbai    | 25000.00     |
| 106      | Customer1     | Hyderabad | 15000.00     |
+-----+-----+-----+-----+
6 rows in set (0.00 sec)

mysql>
mysql>
mysql> -- Q7
mysql> SELECT dept_name,
->         COUNT(*),
->         AVG(salary),
->         MAX(salary)
-> FROM employees e
-> JOIN departments d
-> ON e.dept_id = d.dept_id
-> GROUP BY dept_name;
+-----+-----+-----+-----+
| dept_name | COUNT(*) | AVG(salary) | MAX(salary) |
+-----+-----+-----+-----+
| CSE      | 4       | 72500.000000 | 90000.00     |
| ECE      | 2       | 65000.000000 | 70000.00     |
| EEE      | 2       | 47500.000000 | 50000.00     |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)

```

Screenshot 9: Execution of Q6 (Joining Customers & Orders) and Q7 (GROUP BY Department with aggregate functions).

10. Step 10: Queries Q8 (Filtered Department Join) & Q9 (Customer Total Spend Ranking)

Q8 filters CSE employees earning over 50,000. Q9 computes total purchase amounts per customer ordered in descending order.

```

mysql> -- Q8
mysql> SELECT emp_name,
->         salary,
->         dept_name
-> FROM employees e
-> JOIN departments d
-> ON e.dept_id = d.dept_id
-> WHERE salary > 50000
-> AND dept_name = 'CSE';
+-----+-----+-----+
| emp_name | salary | dept_name |
+-----+-----+-----+
| Rahul    | 80000.00 | CSE       |
| Arjun    | 65000.00 | CSE       |
| Priya    | 55000.00 | CSE       |
| Vijay    | 90000.00 | CSE       |
+-----+-----+-----+
4 rows in set (0.00 sec)

mysql>
mysql>
mysql> -- Q9
mysql> SELECT customer_name,
->         SUM(total_amount) AS TotalPurchase
-> FROM customers c
-> JOIN orders o
-> ON c.customer_id = o.customer_id
-> GROUP BY customer_name
-> ORDER BY TotalPurchase DESC;
+-----+-----+
| customer_name | TotalPurchase |
+-----+-----+
| Customer1     | 85000.00     |
| Customer2     | 55000.00     |
| Customer3     | 45000.00     |
+-----+-----+
3 rows in set (0.00 sec)

```

Screenshot 10: Execution of Q8 (CSE department filtered by salary > 50,000) and Q9 (Top spending customers).

11. Step 11: Queries Q10 (Error Handling), Q11 (COMMIT), and Q12 (ROLLBACK)

Q10 demonstrates MySQL error handling when joining non-existent tables. Q11 and Q12 illustrate basic TCL commands (`COMMIT` and `ROLLBACK`).

```

mysql> -- Q10
mysql> SELECT student_name,
->         course_name,
->         faculty_name
-> FROM students s
-> JOIN enrollments e
-> ON s.student_id = e.student_id
-> JOIN courses c
-> ON e.course_id = c.course_id
-> JOIN faculty f
-> ON c.faculty_id = f.faculty_id;
ERROR 1146 (42S02): Table 'dbms_joins_transactions.students' doesn't exist
mysql>
mysql>
mysql> -- Q11
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> INSERT INTO accounts
-> VALUES (104, 'Anil', 25000);
Query OK, 1 row affected (0.00 sec)

mysql>
mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql>
mysql> -- Q12
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> INSERT INTO accounts
-> VALUES (105, 'Ravi', 10000);
Query OK, 1 row affected (0.00 sec)

mysql>
mysql> ROLLBACK;
Query OK, 0 rows affected (0.00 sec)

```

Screenshot 11: Execution of Q10 (Missing table error handling), Q11 (Transaction COMMIT), and Q12 (Transaction ROLLBACK).

12. Step 12: Queries Q13 (Successful Fund Transfer) & Q14 (Failed Fund Transfer Start)

Q13 demonstrates an atomic fund transfer between account 101 and 102. Q14 initiates a transfer attempt that encounters a target account error.

```
mysql> -- Q13
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> UPDATE accounts
  -> SET balance = balance - 5000
  -> WHERE account_no = 101;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> UPDATE accounts
  -> SET balance = balance + 5000
  -> WHERE account_no = 102;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql>
mysql> -- Q14
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> UPDATE accounts
  -> SET balance = balance - 3000
  -> WHERE account_no = 101;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> UPDATE accounts
  -> SET balance = balance + 3000
```

Screenshot 12: Execution of Q13 (2-step account balance transfer with COMMIT) and Q14 initiation.

13. Step 13: Completing Q14 (ROLLBACK on Error) & Query Q15 (SAVEPOINT & Partial Rollback)

Q14 reverts failed changes. Q15 sets a `SAVEPOINT s1`, performs a second update, rolls back specifically to `s1`, and commits the initial state.

```
-> WHERE account_no = 999;
Query OK, 0 rows affected (0.00 sec)
Rows matched: 0  Changed: 0  Warnings: 0

mysql>
mysql> ROLLBACK;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql>
mysql> -- Q15
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> UPDATE accounts
  -> SET balance = balance + 2000
  -> WHERE account_no = 101;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> SAVEPOINT s1;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> UPDATE accounts
  -> SET balance = balance - 1000
  -> WHERE account_no = 101;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> ROLLBACK TO s1;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)
```

Screenshot 13: Completion of Q14 ROLLBACK and execution of Q15 (SAVEPOINT s1 with ROLLBACK TO s1).